Capitolo 1

Algoritmi su Grafi

1.1 Visitare i Grafi

Esercizio 1.1.1 #Albero #DFS

Dato un grafo $G = (V, E)$ non orientato, scrivere una procedura che verifichi se G è un albero. Dare la definizione di albero. Dare lo pseudocodice dell'algoritmo e studiare la complessità in tempo dell'algoritmo proposto.

Si definisce Albero un grafo composto da una sola componente connessa e senza cicli.

Elaborazione Soluzione: Possiamo vedere se il grafo G è un albero tramite l'algoritmo DFS verificando che consista di una sola componente connessa e che non abbia archi *backward*. Eseguiamo quindi una DFS modificata su G alla ricerca di archi all'indietro e contiamo le componenti connesse.

Si ricorda che all'interno di un grafo non orientato abbiamo 2 tipologie di archi:

1) **Dell'Albero**: Sono gli archi dell'albero di copertura generato dalla DFS. Si hanno quando il vertice raggiunto è bianco.
2) **All'Indietro**: Collegano un vertice ad un suo antenato, quindi si hanno quando il vertice raggiunto è grigio e non è il padre del nodo visitato. Infatti, in un grafo non orientato tutti gli archi sono visitati due volte, una volta per ciascuna direzione.

In un grafo non orientato non ci sono nodi neri da poter raggiungere, perché se un nodo diviene nero significa che ogni arco a lui incidente è stato visitato dalla DFS.

Pseudocodice: Supponiamo che il grafo $G = (V, E)$ sia memorizzato con Liste di Adiacenza.

```
ModDFS(Grafo G)
for ogni u ∈ G.V do
    u.colore = WHITE
    u.π = NULL
bool noCiclo = TRUE
int card = 0
// Se card = |V| dopo una sola invocazione di ModDFSVisit, abbiamo una sola componente connessa
noCiclo, card = ModDfsVisit(G, 1, TRUE, 0)
// La Visita inizia dal vertice memorizzato in posizione 1
if noCiclo == TRUE and card = |V| then
    return: TRUE
else
    return: FALSE

ModDfsVisit(Grafo G, int u, bool noCiclo, int card)
u.colore = GRAY
for ogni v ∈ Adj(u) do
    if v.colore == WHITE then
        card++
        v.π = u
        ModDfsVisit(G, v, noCiclo, card)
    else
        // Grafo non orientato, c'è un ciclo solo se il nodo raggiunto v è grigio ed è diverso dal padre di u
        if v.colore = GRAY And v ≠ u.π then
            return: (noCiclo = FALSE, card)
u.colore = BLACK
return: (noCiclo, card)
```

Lancio dell'algoritmo: ModDfsVisit(G, u, $false$, 0)

Complessità: Il caso pessimo si verfica quando G è un albero, perchè dobbiamo visitarlo tutto prima di concludere. In tal caso, eseguiamo la DFS su ogni vertice, quindi il costo è quello di una DFS del grafo G: $O(V + E)$. Attenzione si possono anche toccare rei vertici neri, ma l'arco che li raggiunge è già stato battezzato in T o in B e quindi non ci interessano.

Esercizio 1.1.2 #BFS

Dato un grafo $G = (V, E)$ orientato, non pesato, e due vertici $s, t \in V$, trovare la distanza da s a t in G.

- Dare la definizione di distanza
- Dare lo pseudo codice dell'algoritmo
- Discutere la complessità dell'algoritmo proposto.

Si definisce distanza da s a t in un grafo orientato e non pesato la lunghezza (numero di archi) del cammino più breve da s a t.

Elaborazione Soluzione: L'algoritmo è dunque una modifica della procedura BFS che termina quando il vertice t è inserito in coda.

```
Mod-BFS(Grafo G, Vertice s, Vertice t)
// Inizializzazione
Coda Q = ∅
for ogni v ∈ G.V do
    d(v) = +∞
    color(v) = WHITE
    π(v) = NULL
d(s) = NULL
ENQUEUE(Q, s)
trovato = FALSE
color(s) = GRAY
while Q ≠ NULL and trovato == FALSE do
    u = DEQUEUE(Q)
    for ogni v ∈ Adj(u) do
        if color(v) == WHITE then
            if v == t then
                trovato = TRUE, d(v) = d(u) + 1
            else
                π(v) = u, color(v) = GRAY, d(v) = d(u) + 1
                ENQUEUE(Q, v)
    color(u) = BLACK
return: d(t)
```

Complessità e Correttezza:

La complessità in tempo dell'algoritmo è:

- $O(V + E)$ se G è rappresentato da liste/vettore di adiacenza
- $O(V^2)$ se G è rappresentato da matrice di adiacenza

L'unica differenza è nell'implementazione del commando For ogni $v \in Adj(u)$, che nel caso della matrice delle adiacenze diventa For $i = 1, \ldots, |V|$

Sappiamo che l'algoritmo è corretto per le proprietà della visita in ampiezza. Quando troviamo t per la prima volta lo abbiamo trovato con il cammino più breve che parte da s.

Esercizio 1.1.3 #Sorgente #AlberoCopertura #EtichettamentoArchi #VettoreAdiacenze #DFS

Dato un grafo $G = (V, E)$ orientato, dare lo pseudo-codice della Depth First Search (DFS) che restituisce un albero di copertura di G radicato in s e restituisce anche il tipo di ciascun arco in E.
Studiare la complessità in tempo dell'algoritmo proposto; si dica se il tipo degli archi dipende dalla sorgente s.

Elaborazione Soluzione: Scriviamo la DFS in modo che essa etichetti i tipi di archi presenti nel grafo $G = (V, E)$.

Pseudocodice: Supponiamo che il grafo $G = (V, E)$ sia memorizzato con Vettore di Adiacenza.

```
DFS(Grafo G)
for ogni u ∈ G.V do
    u.colore = WHITE
    u.π = NULL
time = 0
for ogni u ∈ G.V do
    if u.colore == WHITE then
        time = DfsVisit(G, u, time)

DfsVisit(Grafo G, Vertice u, int time)
time = time + 1
u.d = time
u.coloreGRAY
for [G.V[u]] ≤ i ≤ [G.V[u + 1] − 1] do
    v = G.E[i]
    if v.colore == WHITE then
        v.π = u
        G.E[i].type = "Albero" // Supponiamo in E ci sia il campo type per il tipo di arco
        time = DfsVisit(G, v, time)
    else
        if v.colore == GRAY then
            G.E[i].type = "Indietro"
        else
            if u.d < v.d then
                G.E[i].type = "Avanti"
            else
                G.E[i].type = "Trasversale"
u.colore = BLACK
time = time + 1
u.f = time
return: time
```

Complessità: Il costo è quello di una DFS: $O(V + E)$.

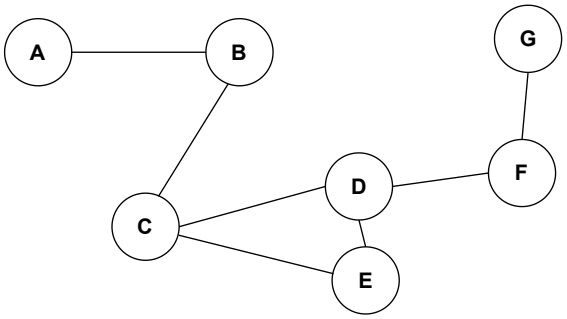
Per concludere, l'esercizio chiedeva di discutere se il tipo degli archi dipende dalla sorgente s. L'albero di copertura dipende dalla sorgente. Basta osservare che in funzione della sorgente s da cui lanciamo la DFS, i tempi di scoperta varieranno. Da questa osservazione, possiamo concludere che il tipo di arco dipende dal nodo sorgente s.

Esercizio 1.1.4 #BFS #DiametroGrafo

Dato un grafo connesso $G = (V, E)$ non orientato, memorizzato tramite liste di adiacenza, trovarne il diametro (la distanza massima fra tutte le coppie dei vertici in G).

- Dare lo pseudo codice dell'algoritmo.
- Discutere la complessità dell'algoritmo proposto.

Esempio:



Diametro: 5

$P_1 : A \to B \to C \to D \to F \to G$

Q: Perchè il diametro non ha valore 6?

$P_2 : A \to B \to C \to E \to D \to F \to G$

A: il cammino P_2 da A a G non è il cammino minimo.

Elaborazione Soluzione: Il diametro è la distanza massima fra le distanze delle coppie dei nodi in G. Formalmente:

$$diam(G) = max_{(u,v)\in V^2}\{dist(u,v)\}$$

Poiché si parla di distanza, sicuramente la visita in ampiezza BFS ci sarà utile. Tuttavia la BFS, fissata la sorgente $s \in V$, $BFS(G,s)$ calcola la distanza di tutti i vertici raggiungibili da s in G. Formalmente, BFS calcola

$$dist(s,v) \quad \forall\ v \in V$$

Pertanto, per calcolare il diametro di un grafo G possiamo sfruttare la visita in ampiezza per calcolare tutti cammini minimi **da ogni sorgente**, quindi per ogni nodo del grafo, a tutti i nodi raggiungibili da essa e memorizzare il massimo cammino computato:

$$\forall s \in V : compute\{dist(s,v)\ \forall\ v \in V\}$$

Realizziamo una versione della BFS che viene eseguita per ogni vertice del grafo. Restituiamo non solo il diametro (come richiesto dall'esercizio) ma anche una coppia la cui distanza è pari al diametro,

Pseudocodice: Supponiamo che il grafo $G = (V, E)$ sia memorizzato con Liste di Adiacenza.

```
get_diametro(Grafo G)
max = −∞, pair_1=pair_2=-1
for ogni v ∈ G.V do
    Coda Q = ∅
    // re-inizializziamo le distanze
    dist[1, ..., |V|] = −1
    dist[v] = 0; // Quella del primo vertice è 0
    color[1, ..., |V|] = WHITE // inizializziamo i colori
    Q.ENQUEUE(v)
    while Q ≠ ∅ do
        u = Q.DEQUEUE()
        for ∀k ∈ G.Adj[u] do
            if k.color == WHITE then
                k.color = GRAY
                dist[k] = dist[u] + 1
                if dist[k] > max then
                    max = dist[k], pair_1 = v, pair_2 = k
                Q.ENQUEUE(k)
        u.color = BLACK
return: max, pair_1, pair−2
```

Complessità: Dato che il primo **for** scorre tutte i nodi in V, abbiamo una BFS ripetuta per ogni nodo del grafo G. Segue che il costo dell'algoritmo è $O(V \cdot (V + E))$; ricordiamo che il grafo è memorizzato con liste di adiacenza.

Esercizio 1.1.5 #DFS

Dato un grafo G orientato ed un nodo $u \in G$, trovare i nodi raggiungibili da u in G.

- Descrivere l'algoritmo
- Dare lo pseudocodice dell'algoritmo
- Studiare la complessità dell'algoritmo proposto

Elaborazione Soluzione: L'esercizio ci richiede di trovare la raggiungibilità di $R(u)$, ossia tutti i nodi nell'insieme $R(u)$.
Ossia, dato un grafo $G = (V, E)$ e un nodo u, trovare tutti nodi in $v \in V$ per cui esiste un cammino p da u a v ($u \overset{p}{\rightsquigarrow} v$).

Una soluzione è invocare una DFS modificata a partire dal nodo u per visitare $R(u)$.

Pseudocodice: Supponiamo che il grafo $G = (V, E)$ sia memorizzato con Liste di Adiacenza.

```
DFS(Grafo G, Vertice u)
for ogni w ∈ G.V do
    w.color = WHITE
    w.π = NULL // il padre
List L = {u} // Lista in cui mettiamo i nodi
DfsVisit(G, u, L) // Visitiamo solo ciò che è raggiungibile da u
return: L

DfsVisit(Grafo G, Vertice u, List L)
u.color = GRAY
for ogni v ∈ adj[u] do
    if v.color == WHITE then
        APPEND(L, v) // Inseriamo il nodo nella lista
        v.π = u
        DfsVisit(G, v, L)
u.color = BLACK
```

Complessità: L'algoritmo proposto ha complessità è equivalente a quella della DFS: $O(V + E)$, infatti nel caso peggiore ogni nodo è raggiungibile dalla sorgente u.

Esercizio 1.1.6 #BFS #DueColorabile

Dato un non-orientato grafo G, esso è k-colorabile se ad ogni vertice $v \in V$ è possibile assegnare un colore $C = \{1, \ldots, k\}$ in modo tale che per ogni coppia di nodi adiacenti $i, j \in V$, ovvero $(i, j) \in E$, il colore di i sia diverso dal colore di j.

1. Scrivere un algoritmo che dica se un grafo G dato in input è 2-colorabile.
2. Discutere la complessità della soluzione proposta.

Dato un grafo G, 2-colorarlo significa assegnare un colore ad ogni nodo in modo che ogni coppia di nodi adiacenti siano di colore diverso. Ricordiamo che G è 2-colorabile se e solo se non ci sono cicli di lunghezza dispari. Ossia, se non ci sono archi backward che collegano due nodi in due livelli adiacenti del grafo di copertura. Quindi per risolvere il problema possiamo colorare l'albero di copertura per livelli con colori alternanti.

Elaborazione Soluzione: Innanzitutto assumiamo che l'oggetto vertice abbia un campo 2COL il quale conterrà il colore del nodo. Il campo 2COL sarà di tipo booleano, dove TRUE significherà blu, mentre FALSE rosso. Con questo trucco è più facile assegnare colori diversi alternanti ai nodi dell'albero di copertura. Inoltre indichiamo un nodo mai visitato come un nodo la cui distanza dalla sorgente è ∞.

Pertanto il problema della 2-colorazione richiede di assegnare i colori ai nodi in modo che

$$\forall v \in V, \forall u \in G.Adj[v], v.\text{2COL} \neq u.\text{2COL}$$

```
is_2-colorabile(Grafo G, Sorgente s)
Coda Q = ∅
for ogni v ∈ G.V − {s} do
    d[v] = ∞
d[s] = 0
s.2COL = TRUE
Q.ENQUEUE(s)
while Q ≠ ∅ do
    v = Q.DEQUEUE()
    for ∀u ∈ G.Adj[v] do
        if d[u] == ∞ then
            d[u] = d[v] + 1
            u.2COL = NOT(v.2COL)
            Q.ENQUEUE(u)
        else
            if v.2COL == u.2COL then
                return: "non-colorabile"
return: "colorabile"
```

Funzionamento: L'algoritmo proposto non fa altro che sfruttare la visita in ampiezza per colorare il grafo controllando se esiste un conflitto. In particolare, se il nodo viene scoperto per la prima volta gli viene assegnato il colore opposto del padre tramite il comando $u.2\text{COL} = \text{NOT}(v.2\text{COL})$, viceversa, se il nodo era già stato scoperto dalla visita in precedenza si procede a controllare se esiste un conflitto verificando $v.2\text{COL} == u.2\text{COL}$. Se il controllo da esito positivo il grafo G non ammette 2-colorazione. Se non si incontra alcun conflitto si conclude con "colorabile".

complessità: Equivale a quella della BFS; pertanto assumendo di usare liste di adiacenza avremo un costo in tempo pari a $O(V + E)$.

Esercizio 1.1.7 #DFS #ComponenteConnessa

Dato un grafo $G = (V, E)$ non orientato, trovare la componente connessa di cardinalità massima.

- Dare la definizione di componente connessa;
- Descrivere a parole un algoritmo per risolvere l'esercizio;
- Dare lo pseudo codice dell'algoritmo;
- Discutere la complessità dell'algoritmo proposto.

Una componente connessa di un grafo è un insieme di vertici che hanno la stessa visibilità, ossia per ogni coppia di vertici della componente connessa esiste un cammino che li connette. Una componente connessa non è estensibile, ossia è un insieme massimale con la proprietà che i suoi vertici sono mutuamente raggiungibili.

Elaborazione Soluzione: L'algoritmo qui proposto è una versione modificata della DFS che tiene traccia della componente connessa trovata di dimensione maggiore.

Pseudocodice:

```
DFS(Grafo G)
for ogni u ∈ G.V do
    u.colore = WHITE
    u.predecessore = NULL
sizecc = 0
rootcc = 0
Set Sol = ∅
for ogni u ∈ G.V do
    if u.colore == WHITE then
        Set tmpSol = {u} // Per ora c'è solo u
        int tmpSize = 1 // La sua dimensione della componente connessa
        DfsVisit(G, u, *tmpSol, *tmpSize) // * è passaggio per riferimento
        if tmpSize > sizecc then
            rootcc = u
            sizecc = tmpSize
            Sol = tmpSol
    color(u) = BLACK
return: (rootcc, sizecc, Sol)

DfsVisit(Grafo G, Nodo u, Set Pointer * tempSol, int Pointer * tmpSize)
u.colore = GRAY
for ogni v ∈ Adj[u] do
    if v.colore == WHITE then
        color(v) = GRAY
        tmpSize++
        tmpSol = tmpSol ∪ v
        DfsVisit(G, v, *tmpSize, *tmpSol)
color(u) = BLACK
```

Complessità:

- $O(V + E)$ se si utilizza una lista/vettore delle adiacenze
- $O(V^2)$ se si utilizza una matrice delle adiacenze

CP: fino qui **Esercizio 1.1.8** #BFS #LunghezzaCicli

Dato un grafo $G = (V, E)$ non orientato, determinare quanti cicli di lunghezza pari e cicli di lunghezza dispari ci sono.

- Descrivere a parole un algoritmo per risolvere l'esercizio;
- Dare lo pseudo codice dell'algoritmo;
- Discutere la complessità dell'algoritmo proposto.

All'interno di un grafo non orientato abbiamo 2 tipologie di archi: Quelli *dell'albero* di copertura generato dalla BFS, si hanno quando il vertice raggiunto è *bianco*; e quelli *all'indietro* che collegano un vertice ad un suo antenato, quindi si hanno quando il vertice raggiunto è *grigio*.

In un grafo non orientato non ci sono nodi neri da poter raggiungere, perché se un nodo diviene nero significa che ogni sotto-grafo a lui connesso è stato visitato dalla BFS. Essendo il grafo non orientato non ci sono archi orientati entranti in un nodo nero che ne permettano il raggiungimento da parte di altri nodi una volta che è divenuto nero.

Elaborazione Soluzione: Quando si raggiunge un vertice grigio stiamo raggiungendo un nodo già esplorato, significa che vi è un ciclo, esaminando le distanze dalla sorgente dei due vertici possiamo determinare se il ciclo è pari o dispari. Poiché esploriamo in ampiezza, le distanze differiscono al più di 1; se sono uguali, il ciclo è dispari, se sono diverse è pari.

Pseudocodice:

```
BFS-Cicli(Grafo G, Vertice s)
pari = 0
dispari = 0
for ogni v ∈ G.V − s do
    d(v) = +∞
    color(v) = WHITE
    π(v) = NULL
d(s) = 0
color(s) = GRAY
π(s) = NULL
Coda Q = ∅
ENQUEUE(Q, s)
while Q ≠ NULL do
    u = DEQUEUE(Q)
    for ogni v ∈ G.Adj(u) − {u.π} do
        if color(v) == WHITE then
            d(v) = d(u) + 1
            color(v) = GRAY
            π(v) = u
            ENQUEUE(Q, v)
        else
            if d(v) == d(u) then
                dispari + +
            else
                pari + +
    color(u) = BLACK
return: (pari, dispari)
```

Complessità: Qui stiamo eseguendo una normale BFS, vi abbiamo solo aggiunto un controllo quando il vertice raggiunto non è bianco, quindi grigio; quindi il costo è $O(V + E)$ con liste di adiacenza, qui utilizzate.

Esercizio 1.1.9 #DFS #PozzoUniversale

Dato un grafo orientato $G = (V, E)$ scrivere un algoritmo che dica se vi è un pozzo universale; studiarne la complessità.

Elaborazione Soluzione: Un nodo è un pozzo universale se ha un arco entrante da ciascun altro vertice (universale) e nessun arco uscente (pozzo). Si fa la DFS e dentro la DfsVisit si tiene traccia dei vertici uscenti ed entranti del nodo; alla fine della DFS si cerca nella lista dei nodi quel nodo avente grado entrante 0 ed uscente $|V| - 1$.
Supponiamo che il grafo sia rappresentato con liste di adiacenza.

Pseudocodice:

```
DFS(Grafo G)
for ogni w ∈ G.V do
    w.colore = WHITE
    w.predecessore = null
    w.e = 0 // .e rappresenta il contatore dei vertici entranti
    w.u = 0 // .u rappresenta il contatore dei vertici uscenti
for ogni w ∈ G.V do
    if w.colore == WHITE then
        DfsVisit(G, w)
```

```
DfsVisit(Grafo G, Vertice w)
w.colore = GRAY
for ogni v ∈ adj[w] do
    v.e = v.e + 1 // Aggiorno il numero di vertici entranti ed uscenti
    w.u = w.u + 1
    if v.colore == WHITE then
        v.predecessore = w
        DfsVisit(G, v)
w.colore = BLACK

Cerca_Pozzo(Grafo G)
DFS(G) // Lancio la DFS sul grafo G
for ogni w ∈ G.V do
    if w.u == 0 and w.e == (#G.V − 1) then
        return: w
return: NULL
```

Abbiamo tolto la variabile *time* dalla DFS perché non utile al fine del nostro esercizio.

Complessità: Qui si ha una DFS leggermente modificata, al termine della quale scorriamo l'array V per vedere se c'è un pozzo universale; il costo è quello della DFS più quello della ricerca del pozzo: $\Theta(V+E)+\Theta(V)=\Theta(V+E)$

E' possibile risolvere la ricerca del pozzo universale in $O(|V|)$ anche quando il grafo è rappresentato con matrice delle adiacenze. In tal caso se $M[i,j]=0$ e $i \neq j$, implica che j non può essere un pozzo universale perché non c'è l'arco entrante da i verso j. Se se $M[i,j]=1$, i non può essere pozzo universale perché da un arco uscente. Quindi ad ogni confronto non sulla diagonale si esclude un vertice candidato. Precisamente, si parte da $M(1,2)$. Se $M(i,j)=0$, si esclude j e ci si posta a $M(i,j+1)$ (si incrementa j). Invece se $M(i,j)=1$, si esclude i e ci si posta a $M(i+1,j)$. Si continua fino a che $i=|V|$ or $j=|V|$. L'indice diverso da $|V|$ nella coppia è testato per vedere se è un pozzo universale.

Esercizio 1.1.10 #BFS #CamminiMinimi

Dato un grafo $G=(V,E)$ non pesato (o con pesi costanti), trovare la distanza tra tutti i nodi in G.

- Dare la definizione di distanza;
- Descrivere a parole un algoritmo per risolvere l'esercizio;
- Dare lo pseudo codice dell'algoritmo;
- Discutere la complessità dell'algoritmo proposto.

Si definisce distanza da s a t in un grafo orientato e **non pesato** la lunghezza, intesa come numero di archi, del cammino più breve da s a t.

Elaborazione Soluzione: L'algoritmo risolutivo è una modifica della procedura BFS, questa viene iterata per ogni nodo, ed ogni cammino trovato dalla radice a gli altri nodi va ad aggiornare una matrice D in cui memorizziamo le lunghezze dei cammini; poiché la BFS trova sempre il cammino minimo dalla radice al nodo, dobbiamo solo scrivere nella matrice il valore trovato. Nella pratica stiamo quindi riempiendo la matrice una riga alla volta.

Pseudocodice:

```
Distance-All(Grafo G)
Matrix D[G.V × G.V] = ∅
for ogni v ∈ G.V do
    D = BFS-Distanze(G, v, D) // Aggiorno la Matrice
return: D
```

```
BFS-Distanze(Grafo G, Vertice s, Matrice D)
for ogni v ∈ G.V − s do
    d(v) = +∞
    color(v) = WHITE
    π(v) = NULL
d(s) = 0
color(s) = GRAY
π(s) = NULLL
CodaQ = ∅
ENQUEUE(Q, s)
D[s, s] = 0
while Q ≠ NULL do
    u = DEQUEUE(Q)
    for ogni v ∈ G.Adj(u) do
        if color(v) == WHITE then
            d(v) = d(u) + 1
            color(v) = GRAY
            π(v) = u
            ENQUEUE(Q, v)
            D[s, v] = d(v)
    color(u) = BLACK
return: D
```

Complessità: Abbiamo qui eseguito una BFS per ogni nodo, quindi il costo, con liste di adiacenza qui usate, è $O(V) \cdot O(V + E) = O(V^2 + V \cdot E)$.
A livello spaziale, oltre al costo della lista di adiacenza per memorizzare il grafo G, pari a $O(V + E)$, si aggiunge il costo per la matrice con le lunghezze dei cammini minimi tra i nodi, pari a $O(V^2)$; per un costo spaziale totale di $O(V + E) + O(V^2) = O(V^2)$. L'algoritmo è ottimo in quanto deve restituire V^2 valori e perciò $\Omega(V^2)$.

Esercizio 1.1.11 #BFS #Eccentricità

Dato un grafo $G = (V, E)$ non orientato, non pesato (o con pesi costanti) e dato un nodo v, trovare la sua eccentricità in G (i.e., $e(v) = \max\{d(v, u)|u \in V\}$).

- Descrivere a parole un algoritmo per risolvere l'esercizio;
- Dare lo pseudo codice dell'algoritmo;
- Discutere la complessità dell'algoritmo proposto.

L'eccentricità di un nodo in un grafo è una misura della massima distanza tra quel nodo e qualsiasi altro nodo nel grafo.

Elaborazione Soluzione: Possiamo risolvere questo problema utilizzando l'algoritmo Breadth-First Search (BFS). La BFS ci permette infatti di trovare i percorsi più brevi tra un nodo di partenza e tutti gli altri nodi nel grafo in termini di archi.
Eseguendo BFS a partire da un certo nodo $v \in V$ dato in input e trovando la distanza massima da qualsiasi altro nodo, possiamo determinare l'eccentricità del nodo v ritornando il nodo u con la distanza massima da v, i.e., $max\{d(v, u)|u \in V\}$.

Pseudocodice:

```
find_eccentricity(Grafo G, vertex s)
for ogni v ∈ G.V − s do
    d(v) = +∞
    color(v) = WHITE
    π(v) = NULL
max_d = 0
d(s) = NULL
color(s) = GRAY
π(s) = NULL
Coda Q = ∅
ENQUEUE(Q, s)
while Q ≠ NULL do
    u = DEQUEUE(Q)
    for ogni v ∈ G.Adj(u) do
        if color(v) == WHITE then
            d(v) = d(u) + 1
            if max_d < d(v) then
                max_d = d(v)
            color(v) = GRAY
            π(v) = u
            ENQUEUE(Q, v)
    color(u) = BLACK
return: max_d
```

Complessità:

- $O(V + E)$ se G è rappresentato da liste/vettore di adiacenza
- $O(V^2)$ se G è rappresentato da matrice di adiacenza

Esercizio 1.1.12 #BFS #RaggioGrafo #CentroGrafo

Dato un grafo $G = (V, E)$ non orientato, non pesato (o con pesi costanti), trovarne il raggio ed il centro.

- Dare la definizione di Raggio e Centro di un grafo;
- Descrivere a parole un algoritmo per risolvere l'esercizio;
- Dare lo pseudo codice dell'algoritmo;
- Discutere la complessità dell'algoritmo proposto.

Il raggio di un grafo è la minima tra tutte le distanze massime tra un vertice e tutti gli altri vertici; quindi è la minima delle eccentricità. Il centro di un grafo è il nodo la cui eccentricità è uguale al raggio; quindi quel nodo avente eccentricità minima.

Elaborazione Soluzione: Ci basta calcolare le eccentricità dei nodi, trovare quelle minime e restituire sia tale valore sia le sorgenti da cui partono i cammini; infatti un grafo può avere più centri, tutti di stessa eccentricità, quella minima.

Pseudocodice:

```
Eccentricity-All(Grafo G)
minEccentricity = +∞
for ogni v ∈ G.V do
    ecc(v) = 0 // Inizializzo le Eccentricità di ogni Nodo
for ogni v ∈ G.V do
    ecc(v) = BFS-Eccentricity(G, v)
    if minEccentricity > ecc(v) then
        minEccentricity = ecc(v)
ListL = ∅
for ogni v ∈ G.V do
    if ecc(v) == minEccentricity then
        APPEND(L, v) // Lista con i Centri
return: e, L

BFS-Eccentricity(Grafo G, vertex s)
for ogni v ∈ G.V − s do
    d(v) = +∞
    color(v) = WHITE
    π(v) = NULL
max_d = 0
d(s) = NULL
color(s) = GRAY
π(s) = NULL
CodaQ = ∅
ENQUEUE(Q, s)
while Q ≠ NULL do
    u = DEQUEUE(Q)
    for ogni v ∈ G.Adj(u) do
        if color(v) == WHITE then
            d(v) = d(u) + 1
            if max_d < d(v) then
                max_d = d(v)
            color(v) = GRAY
            π(v) = u
            ENQUEUE(Q, v)
    color(u) = BLACK
return: max_d
```

Complessità: Abbiamo eseguito la ricerca della eccentricità per ogni nodo. Tale ricerca è una BFS con l'aggiunta di un **if**, quindi ha costo $O(V+E)$, facendola per ogni nodo abbiamo un costo di: $O(V^2+V\cdot E)$.

Esercizio 1.1.13 #BFS

Dato un grafo $G = (V, E)$ non diretto e connesso, trovare se esistono due vertici x e y in G connessi da due cammini, cioè connessi da due cammini che non condividono né archi né vertici (esclusi gli estremi x e y del cammino stesso).
Descrivere un algoritmo che risolve il problema e darne lo pseudocodice.
Studiare la complessità dell'algoritmo proposto.

Elaborazione Soluzione: Il problema dice: "trovare se esistono due vertici x e y in G connessi da due cammini". Questo significa semplicemente verificare se vi è o no un ciclo qualsiasi nel grafo; infatti se esiste un ciclo allora i vertici nel ciclo sono connessi da due cammini diversi. Si trova un ciclo in un grafo non orientato tramite una DFS od una BFS quando un vertice raggiunto dalla visita è già stato esplorato.

Supponiamo invece che il problema chieda di verificare che: dati due nodi x e y in G, se essi, nello specifico, sono connessi da due cammini distinti.
In questo caso, per trovare la soluzione, utilizziamo la BFS a partire da x; quando raggiungeremo y avremo il cammino più breve che li connette e se lo raggiungiamo di nuovo avremo allora un altro cammino distinto.

Pseudocodice:

```
BFS-2-paths(Grafo G, Nodo x, Nodo y)
// G contiene V, E ed adj
bool 2paths = False
for ogni u ∈ G.V do
    u.color = WHITE
x.color = GRAY
Coda Q = ∅
ENQUEUE(Q, x)
while IS_EMPTY(Q) == False do
    Nodo u = DEQUEUE(Q)
    for ogni v ∈ G.adj[u] do
        if v.color == WHITE then
            v.color = GRAY
            ENQUEUE(Q, v)
        else
            if v == y then
                2paths = True
    u.color = BLACK
DELETE(Q)
return: 2paths
```

Complessità: L'algoritmo esegue una BFS con l'aggiunta di un **else** nel caso in cui il colore del vertice che si raggiunge non è bianco. In quel caso si controlla che sia o no y; se lo è allora abbiamo i due cammini distinti. Il costo è quindi quello della BFS, cioè $O(V + E)$.

Osservazione: L'algoritmo era realizzabile anche con una DFS.

Esercizio 1.1.14 #DFS

Sia $G = (V, E)$ un grafo non diretto e non connesso: trovare quanti archi si debbono aggiungere per rendere connesso G.
Descrivere un algoritmo che risolve il problema e darne lo pseudocodice.
Studiare la complessità dell'algoritmo proposto.

Elaborazione Soluzione: Sappiamo che gli algoritmi BFS e DFS possono essere utilizzati per la ricerca di componenti connesse. Per connettere un grafo sconnesso servono tanti archi quanti il numero di componenti connesse meno uno.

Pseudocodice:

```
DFS-Comp-Conn(Grafo G)
for ogni u ∈ G.V do
    color(u) = WHITE
int i = 0 // numero di componenti
for ogni u ∈ G.V do
    i++
    if color(u) = WHITE then
        DfsVisit(G, u)
return: i − 1

DfsVisit(Grafo G, Vertice u)
color(u) = GRAY
for ogni v ∈ Adj[u]} do
    if color(v) = WHITE then
        v.predecessore = u
        DfsVisit(G, v)
color(v) = BLACK
```

Complessità: Stiamo qui eseguendo una normale DFS, quindi il costo è quello, $\Theta(V + E)$.

1.2 Operazioni sui Grafi

Esercizio 1.2.1 #DFS #OrdinamentoTopologico #GrafoDirettoAciclico

Dato un grafo $G = (V, E)$ orientato ed una sorgente r dalla quale si raggiungono tutti i vertici in V, restituire, se esiste, l'ordine dei vertici che soddisfa l'ordinamento topologico.

- Dare la definizione di ordinamento topologico;
- Dire sotto quali condizioni esiste l'ordinamento topologico;
- Dare lo pseudo codice dell'algoritmo ('algoritmo deve restituire False se l'ordinamento non è possibile);
- Discutere la complessità dell'algoritmo proposto.

Elaborazione Soluzione: E' possibile trovare l'ordinamento topologico solo se il grafo è un Directed Acyclic Graph (DAG). L'ordinamento topologico garantisce che, esiste una linearizzazione dei vertici tale che posti i vertici sulla linea per ciascun arco il vertice da cui esso parte precede nell'ordinamento il vertice destinazione dell'arco stesso.

L'algoritmo che risolve il nostro esercizio, SORT-TOP, ritorna la lista L dei vertici in ordine decrescente di fine visita (il pozzo del nostro grafo ha il minimo tempo di visita). L'algoritmo ritorna FALSE se non esiste l'ordinamento topologico, ossia se trova un arco di tipo backward. In tal caso, esiste un ciclo e pertanto il grafo non è un DAG.

Pseudocodice:

```
DFS(Grafo G)
for ogni u ∈ G.V do
    color(u) = WHITE
    u.predecessore = NULL
List L = ∅ // Lista ordinata dei vertici visitati
DFSVisit(G, u, 0, L)

DFSVisit(Grafo G, Vertice u, int time, List L)
time = time + 1
u.d = time
color(u)GRAY
for ogni v ∈ Adj[u]} do
    if color(v) = WHITE then
        v.predecessore = u
        DFSVisit(G, v, time, L)
    else
        if color(v) = GRAY then
            // Abbiamo un arco all'indietro, quindi non è un DAG
            return: False
color(v) = BLACK
time = time + 1
u.f = time
INSERT(u, L) // Inserimento in testa
return: L

sort-top(Grafo G)
result = DFS(G)
if result ≠ False then
    return: result
    // Si ritorna la lista dei vertici ordinati topologicamente
else
    return: "Vi è un Ciclo"
```

Complessità:

- Abbiamo la complessità della DFS su lista di adiacenza: $O(V + E)$

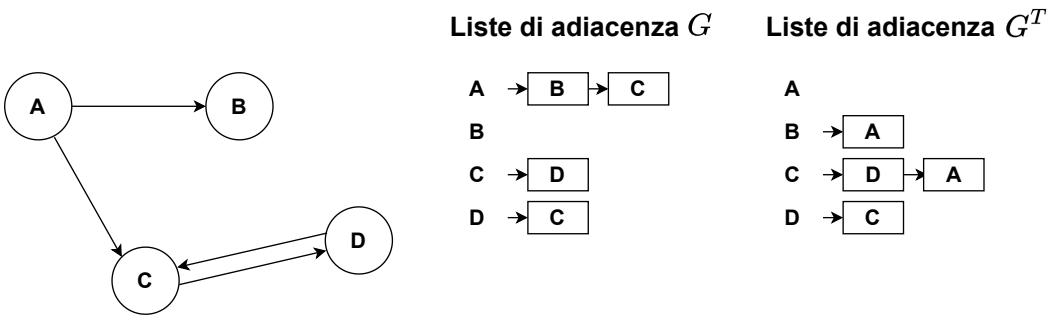
- Poichè dal vertice r si raggiungono tutti gli altri vertici è di fatto sufficiente invocare la DfsVisit a partire da r.

Esercizio 1.2.2 #ListeAdiacenza #GrafoTrasposto #GrafoDiretto

Dato un grafo $G = (V, E)$ orientato, memorizzato tramite liste di adiacenza, trovarne il trasposto G^T.

- Dare lo pseudo codice dell'algoritmo.
- Discutere la complessità dell'algoritmo proposto.

Esempio:



Elaborazione Soluzione: Per ogni coppia di nodi $v, u \in V$ viene invertita la relazione che li lega.

Pseudocodice

Reverse_List_Adj(**Grafo** G)
inizializza G^T
for $\forall v \in G.V$ **do**
  for $\forall u \in G.adj[v]$ **do**
    $G^T.adj[u]$.APPEND(v)
return: G^T

Funzionamento:

- per **inizializza** G^T si intende la creazione di un dizionario le cui chiavi sono i vertici della grafo G, e i cui campi sono vuoti; ovvero ogni lista delle adiacenze ha la sola testa inizializzata.
- La funzione APPEND() viene utilizzata per inserire in testa l'ultimo elemento che si è letto nella rispettiva lista.

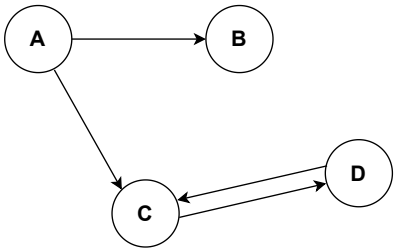
Complessità: Il primo ciclo **for** itera sopra ogni vertice di $G.V$, segue che il suo costo è $O(V)$. Il secondo ciclo **for** visita per ogni nodo tutti i suoi archi uscenti esattamente una volta; pertanto ci costa $O(E)$. Segue che la complessità finale dell'algoritmo proposto è $O(V + E)$.

Esercizio 1.2.3 #MatriceAdiacenza #GrafoTrasposto #GrafoDiretto

Dato un grafo $G = (V, E)$ orientato, memorizzato tramite matrice di adiacenza, trovarne il trasposto G^T.

- Dare lo pseudo codice dell'algoritmo.
- Discutere la complessità dell'algoritmo proposto.

Esempio:



Matrice di adiacenza G

	A	B	C	D
A	0	1	1	0
B	0	0	0	0
C	0	0	0	1
D	0	0	1	0

Liste di adiacenza G^T

	A	B	C	D
A	0	0	0	0
B	1	0	0	0
C	1	0	0	1
D	0	0	1	0

Elaborazione Soluzione: Scriviamo uno pseudocodice che implementa una generica trasposizione di una matrice. Nello specifico, si devono andare a scambiare le colonne della matrice di partenza (G), con le righe della matrice di destinazione (G^T).

Pseudocodice:

Reverse_Matrix_Adj(**Grafo** G)
inizializza G^T
for $i = 0$ **to** $|V| - 1$ **do**
  for $j = 0$ **to** $|V| - 1$ **do**
    $G^T[j][i] = G[i][j]$
return: G^T

Complessità: Il costo dell'algoritmo segue dall'analisi dei due **for**. Entrambi ci costano $O(V)$, quindi visto che sono innestati il costo finale sarà $O(V^2)$.

Esercizio 1.2.4 #GrafoPotenza #DFS

Dato un grafo $G = (V, E)$ diretto, non pesato e memorizzato con matrice di adiacenza, e dato un intero k, si vuole costruire il grafo potenza G^K; quindi un grafo che ha gli stessi vertici di G e un arco per ogni coppia di nodi a distanza **al più** k in G.
Descrivere un algoritmo che risolve il problema e darne lo pseudocodice.
Studiare la complessità dell'algoritmo proposto.

Elaborazione Soluzione:

Definiamo:

$$L^k_{(i,j)} = \bigvee_{h=1}^{n} \left(L^{k-1}_{(i,h)} \wedge L^1_{(h,j)} \right)$$

Tale formula ci permette di determinare se tra due nodi i e j nel grafo L vi è un cammino di lunghezza k, nella posizione $L^k_{(i,j)}$ della matrice avremo TRUE se esiste l'arco nel grafo potenza e FALSE diversamente.

Supponiamo che la matrice di adiacenza iniziale data abbia anche essa TRUE dove vi è un arco tra due nodi e FALSE dove non vi è.

Per risolvere il nostro problema dobbiamo quindi calcolare tutti i grafi potenza fino a k e sovrapporli per trovare i cammini di lunghezza al più k; quindi fare l'unione di tutti i grafi G^1, G^2, $\cdots$, G^k a formare G^K.

Pseudocodice:

GrafoPotenza(**Matrix** $G[n \times n]$, **int** k)
// G è matrice di adiacenza con TRUE e FALSE
$G^1[n \times n] = G$
for $m = 2$ **to** k **do**
  G^m = OPERATION($G^{(m-1)}$, G^1)// le matrici sono passate per riferimento
return: G^k

```
Operation(Matrix G_{m-1}[n × n], Matrix G_1[n × n])
Matrix G_m[n × n] // Creo una nuova matrice di dimensione n × n
for i = 1 to n do
    for j = 1 to n do
        temp = False
        h = 1
        while h ≤ n and temp == False do
            temp = temp or (G_{m-1}[i, h] and G_1[h, j])
            h++
        G_m[i, j] = temp
return: G_m
```

Complessità: L'algoritmo equivale a fare k volte una moltiplicazione di matrici $n \times n$. Quindi ha costo $O(k \cdot n^3)$

Esercizio 1.2.5 #GradoMedio #DFS

Dato un grafo non orientato $G = (V, E)$, scrivere un algoritmo per determinare il grado medio di G.
Studiare la complessità in tempo dell'algoritmo proposto in funzione della rappresentazione utilizzata per G.

Elaborazione Soluzione: Il grado di un nodo in un grafo non orientato è il numero di archi ad esso incidenti. Per trovare il grado medio di un grafo si devono sommare i gradi di tutti i nodi e dividere tale valore per il numero di nodi del grafo.

Dato che abbiamo la lista di adiacenza del grafo, potremo per ogni nodo della lista misurare della dimensione della lista di adiacenza ad esso collegata per trovare il grado del nodo; il costo per la lettura della lista è di $O(V + E)$.

Pseudocodice:

```
AverageDegree(Grafo G)
int c = 0
for ogni v ∈ G.V do
    for ogni u ∈ G.E[v] do
        c++
return: c/|V|
```

Complessità: Il costo è $O(V + E)$.

1.3 Alberi di Copertura

Esercizio 1.3.1 #Kruskal #AlberoCoperturaCostoMinimo

Dato un grafo connesso $G = (V, E)$ non orientato, i cui archi hanno pesi interi minori od uguali a 3, ossia $w : E \to \{1, 2, 3\}$, trovare l'albero di copertura T_G di costo minimo. Dare lo pseudocodice dell'algoritmo e studiare la complessità in tempo dell'algoritmo proposto.

Elaborazione Soluzione: Il grafo $G = (V, E)$ è caratterizzato dal fatto che la funzione peso è definita come segue: $w : E \to \{1, 2, 3\}$.

Sappiamo che l'algoritmo di Kruskal per il calcolo dell'albero di copertura minimo seleziona ad ogni passo gli archi più leggeri, assicurandosi di non formare cicli. Inizia con un insieme vuoto di archi e poi ordina gli archi in base ai loro pesi. Quindi, itera attraverso gli archi, aggiungendo all'insieme quelli che non creano cicli. L'algoritmo continua finché l'albero di copertura minimo contiene $|V| - 1$ archi.

Dato che l'insieme dei pesi è finito e caratterizzato da soli valori interi, possiamo ottenere un miglioramento delle prestazioni di Kruskal utilizzando l'algoritmo di ordinamento Counting Sort, con il quale si ordinano gli archi in tempo lineare.

Pseudocodice:

```
KruskalCountingSort(Graph G)
List E = Archi di G.E Ordinati in Ordine Decrescente Usando Counting-Sort
List T = ∅ // Lista per gli archi dell'albero di copertura minimo
for ogni v in G.V do
    MAKE-SET(v)
for ogni arco (u, v) in E preso nell'ordine dalla lista E do
    if FIND(u) ≠ FIND(v) then
        T = T ∪ (u, v)
        UNION(u, v)
return: T
```

Complessità: Il tempo di esecuzione di Kruskal è dato dall'uso di Merge-Sort per ordinare $G.E$ e dall'utilizzo della struttura dati ad Insiemi Disgiunti. Per le strutture dati, noi esprimiamo la complessità in tempo come numero di operazione Find e numero di operazioni Union. L'uso del Counting-Sort migliora il tempo dell'ordinamento, perché porta il costo dell'ordinamento a $O(E)$.
Il tempo di esecuzione asintotico pertanto è: $O(E + 2|E|T_{\text{FIND}} + (|V| - 1)T_{\text{UNION}})$.

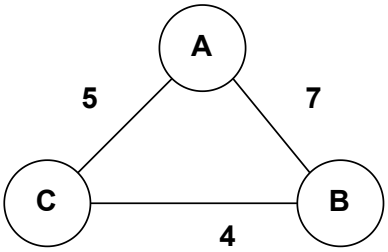
Esercizio 1.3.2 #AlberoCoperturaCostoMinimo #AlberoCoperturaCamminimiCostoMinimo

Si consideri il grafo non orientato $G = (V, E)$ e pesato $w : E \to N$ dove $V = \{A, B, C\}$ e $E = (A, B), (A, C), (B, C)$ (ossia un ciclo con 3 vertici).

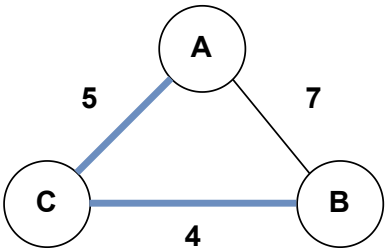
- Determinare i pesi sugli archi in modo che l'albero dei cammini di costo minimo (Albero dei Cammini Minimi) radicato nel vertice A sia diverso dall'albero di copertura di costo minimo (Albero di Copertura Minimo) del grafo G;
- Illustrare i due alberi trovati

Proponiamo qui una possibile soluzione dell'esercizio:

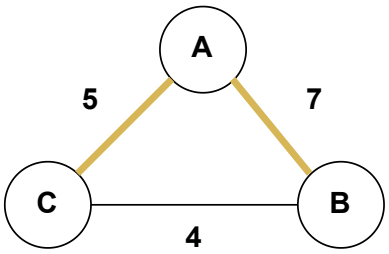
Soluzione:



MST



Cammini Minimi



Esercizio 1.3.3 #Prim #AlberoCoperturaCostoMinimo

Dato un grafo connesso $G = (V, E)$, non orientato, i cui archi hanno pesi interi, trovare la componente connessa di costo minimo (sotto-albero di copertura di costo minimo) T_s radicato in s con esattamente k vertici.
Dare lo pseudocodice dell'algoritmo e studiare la complessità in tempo dell'algoritmo proposto.

Elaborazione Soluzione: L'algoritmo di Prim ha la proprietà per cui gli archi aggiunti all'albero di copertura formano sempre un unico albero; per cui l'albero inizia da una radice arbitraria, qui scelta in s, e cresce fino a coprire tutti i vertici del grafo. Ad ogni passo si aggiunge all'albero l'arco più leggero che connette l'albero ad un vertice isolato, quindi un vertice non ancora connesso all'albero; in questo modo si aggiungono solo vertici "sicuri" per l'albero, poiché non formano cicli. Per prendere sempre l'arco più leggero si mantiene un Min-Heap.

Per via della caratteristica dell'algoritmo a costruire l'albero di copertura a partire dalla sorgente data, è adatto al nostro problema; ci basta eseguirlo controllando poi il numero di vertici inseriti dell'albero. L'algoritmo quindi chiede di troncare l'esecuzione dopo aver inserito k vertici, quindi si ritorna un sotto-albero dell'albero di copertura di costo minimo.

Pseudocodice:

```
Prim(Graph G, Nodo s, FunzionePeso ω, Intero k)
for ogni vertice u ∈ G.V do
    u.key = ∞
    u.π = NULL
s.key = 0
Min-Heap Q = ∅
for ogni vertice u in G.V do
    INSERT(Q, u)
i = 0 // Contatore
while i < k do
    u = EXTRACT-MIN(Q) // Aggiungiamo u all'albero
    i++
    for ogni vertice v ∈ G.adj[u] do
        // Si aggiornano le chiavi dei nodi vicini ad u nell'albero e non ancora estratti da Q
        if v ∈ Q and ω(u, v) < v.key then
            v.π = u
            v.key = ω(u, v)
            DECREASE-KEY(Q, v, ω(u, v))
```

Complessità: L'algoritmo di Prim così modificato ha complessità asintotica $O(V + k \cdot d_{\mathbf{max}} \cdot \log V)$, dove $d_{\mathbf{max}}$ è il max grado dei nodi in G, quando implementato con min-heap.

Esercizio 1.3.4 #Kruskal #AlberoCoperturaCostoMinimo

Dato un grafo non diretto $G = (V, E)$ i cui archi hanno peso 1 o 10, ossia $w : E \to \{1, 10\}$, la Prof. Fati-Na, ha proposto il seguente algoritmo per calcolare l'albero di copertura di costo minimo:

```
MST-UltraShort(Grafo G, Funzione Peso w)
Let E' ⊆ E is the set of edges with w(e) = 1
T = DFS(G' = (V, E'))
return: T
```

L'algoritmo proposto è corretto?
Motivare la risposta: se non è corretto dare un controesempio. Eventualmente correggere l'algoritmo.
Discutere la complessità dell'algoritmo, eventualmente corretto.

L'algoritmo non è corretto. L'eliminazione di archi di costo 10 potrebbe disconnettere dei vertici dal grafo; ad esempio avendo un vertice i cui archi incidenti hanno solo costo 10.

Algoritmo Corretto:

```
KruskalCountingSort(Graph G)
List E = Archi di G.E Ordinati in Ordine Decrescente Usando Counting-Sort
List T = ∅ // Lista con gli Archi dell'albero di copertura minimo
for ogni v in G.V do
    MAKE-SET(v)
for ogni arco (u, v) in E preso nell'ordine dalla lista E do
    if FIND(u) ≠ FIND(v) then
        T = T ∪ (u, v)
        UNION(u, v)
return: T
```

Complessità: Sapendo che i pesi degli archi sono nell'insieme $\{1, 10\}$ possiamo applicare l'algoritmo di Kruskal con counting-sort per migliorarne il costo; infatti la complessità temporale dell'algoritmo di Kruskal è data principalmente dall'ordinamento dei vertici, che qui è $O(E)$. Il tempo di esecuzione asintotico pertanto è $O(E + 2|E|T_{\text{FIND}} + (|V| - 1)T_{\text{UNION}})$.

Esercizio 1.3.5 #Kruskal #AlberoCoperturaCostoMinimo

Dato un grafo non orientato e pesato $G = (V, E)$ e il suo albero di copertura di costo minimo T, si consideri il grafo ottenuto aggiungendo l'arco pesato $e = (u, v)$, ossia $G' = (V, E \cup (u, v))$ (i vertici sono invariati). Sia T' l'albero di copertura di costo minimo per G'. Dire per ogni affermazione se è vera o falsa (giustificando la risposta):

- $e \in T'$ se il peso $w(e)$ è il peso minimo degli archi in G
- $e \in T'$ se il peso $w(e)$ è il peso massimo degli archi in G'
- T' e T differiscono al più per 1 arco

Dare lo pseudocodice di un algoritmo che calcola T' conoscendo G' e T.
Discutere la complessità in tempo dell'algoritmo proposto quando i pesi degli archi sono interi compresi nell'intervallo $[0, 10]$.

Risposte alle Domande. Si fa riferimento solo alla costruzione di T' a partire da T. Si noti che il nuovo arco forma un ciclo C.

- Vero perché sarà il primo arco colorato di blue dall'algoritmo. Solo se C contiene tutti archi del peso uguale a $w(e)$ allora e non è inserito in T'.
- Falso. L'arco di costo massimo $w(e)$ è escluso. Attenzione, si noti che i vertici non sono stati modificati.
- Falso. Ecco un caso in cui T e T' differiscono per due archi. $w(e)$ è l'arco di costo minimo, e entra in T', ma un altro arco esce perchè si forma un ciclo.

Elaborazione Soluzione: Sappiamo che T è un albero di copertura minimo per G, poiché G' differisce da G solo per la aggiunta dell'arco e, possiamo semplicemente aggiungere e a T, creando T_e, per poi fare in modo che T_e resti un albero di copertura minimo, così da avere T'.

Dato l'arco $e = (u, v)$, seguendo il campo padre in T troviamo gli archi che formeranno il ciclo con e. Escludiamo l'arco di peso massimo, aggiungiamo e e dovremmo correggere i padri.

Per risolverlo in modo standard possiamo invocare Kruskal sui $|V| - 1$ archi in T e sul nuovo arco e.

Pseudocodice:

```
KruskalDatoT(Tree T, e)
Lista E = tutti gli archi in T ∪ e
for ogni v in G.V do
    MAKE-SET(v)
for ogni arco (u, v) in E preso nell'ordine dalla lista E do
    if FIND(u) ≠ FIND(v) then
        T = T ∪ (u, v)
        UNION(u, v)
return: T
```

Complessità: $O(V \cdot \log V)$ per ordinare, V UNION, $2|V|$ FIND. Soluzione sub-ottima perché coinvolge archi che non stanno in C.

Esercizio 1.3.6 #AlberoCoperturaCostoMinimo #Kruskal

Dato un grafo non orientato $G = (V, E)$ ed una sorgente s, i cui archi sono pesati con interi compresi nell'intervallo $[0, b]$, progettare un algoritmo per trovare l'albero di copertura T_G di costo minimo.
Discutere la complessità in tempo dell'algoritmo proposto.
Come varia l'albero di copertura se la sorgente scelta varia?
E' possibile dare in tempo costante un upper-bound al costo dell'albero di copertura di T_G? senza eseguire l'algoritmo"

Elaborazione Soluzione: Eseguiamo l'algoritmo di Kruskal ordinando gli archi non con MERGE-SORT ma con COUNTING-SORT.

Pseudocodice:

KruskalCountingSort(**Graph** G, **int** b)
Lista E = COUNTING-SORT($G.E, b$)
List $T = \emptyset$ // Lista con gli Archi dello albero di copertura minimo
for *ogni* v *in* $G.V$ **do**
  MAKE-SET(v)
for *ogni arco* (u, v) *in* E *preso nell'ordine dalla lista* E **do**
  if FIND(u) $\neq$ FIND(v) **then**
    $T = T \cup (u, v)$
    UNION(u, v)
return: T

Complessità: Dato che il costo dell'algoritmo di Kruskal dipende dall'algoritmo di ordinamento usato per ordinare i vertici, il costo passa dal $O(E \cdot \log E)$ di quando si usa MERGE-SORT a $O(E + 2|E| \cdot T_{\text{FIND}} + (|V| - 1) \cdot T_{\text{UNION}})$.

Risposta alla Domanda: Sì in tempo costante è possibile limitare superiormnete il costo massimo dell'albero di copertura. Tale costo è $O(b * (|V| - 1))$.